Technical Design Document (TDD)

Project Name: KidDose / DoseDek

Document Version: 1.0.0

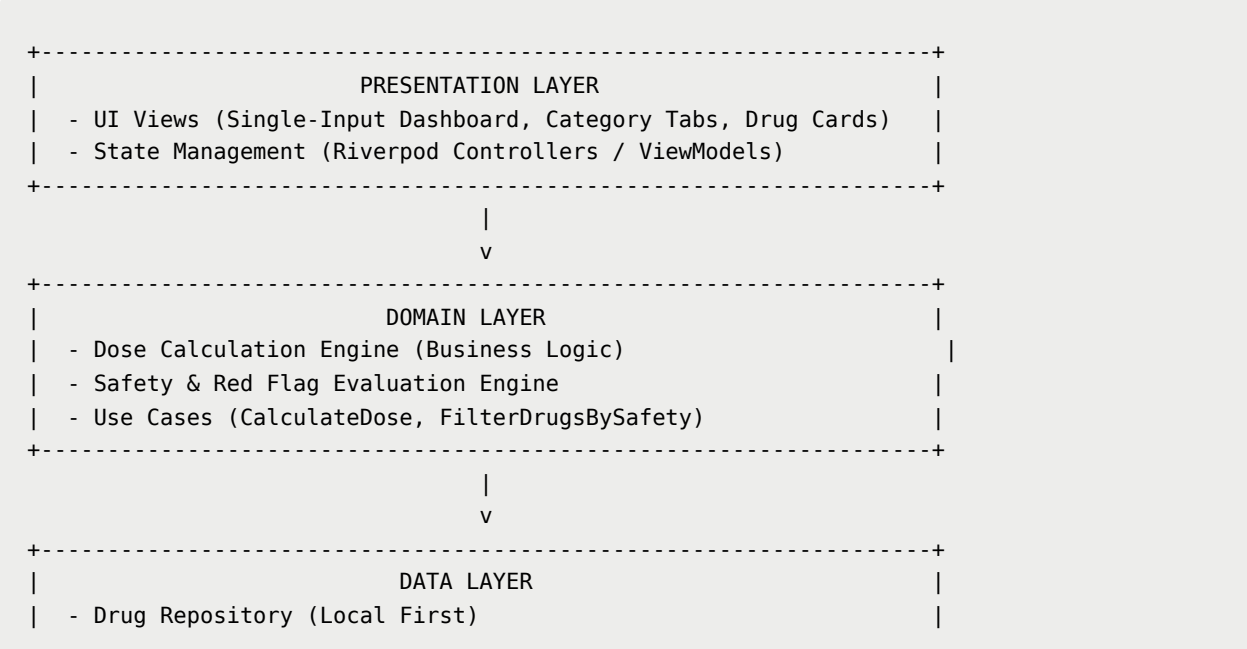
Status: Draft / Technical Architecture

Target Architecture: Cross-Platform Mobile App (Flutter) — Offline First

1. System Architecture Overview

KidDose is designed as a strict **Offline-First, High-Performance Mobile Application**. The system relies on a unidirectional data flow and clean architecture separation to guarantee sub-100ms render response times upon weight input changes.

1.1 High-Level Architecture (Clean Architecture)



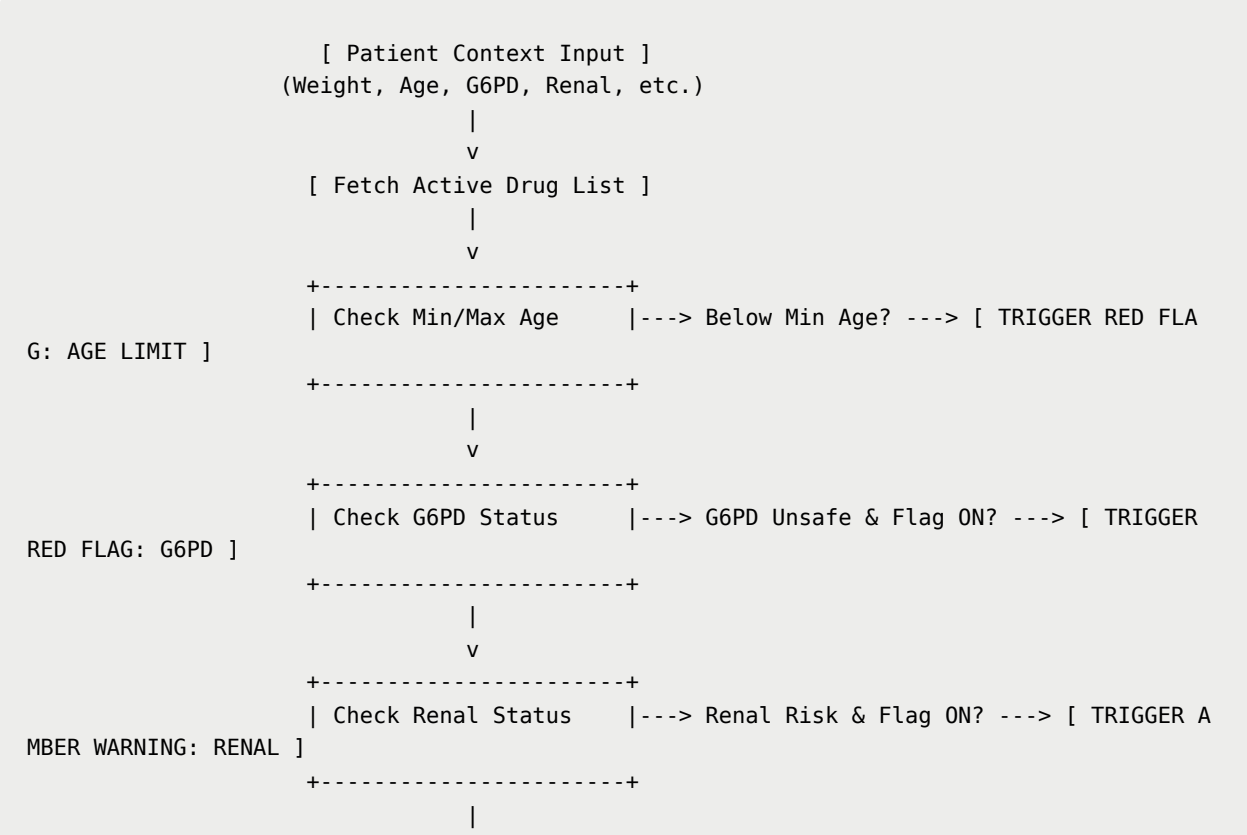
```
|  - Local Storage (Isar / Hive NoSQL DB)                          |
|  - Asset Seeder (Bundled Local JSON Database)                    |
+------------------------------------------------------------------+
```

2. Calculation & Safety Engine Logic

2.1 Mathematical Formulas

1. **Single Mg Dose Calculation:**$$\text{Target Dose (mg)} = \min\left(\text{Weight (kg)} \times \text{Dose}_{\text{std (mg/kg/dose)}}, \text{Max Single Dose (mg)}\right)$$
2. **Liquid Volume Calculation:**$$\text{Volume (mL)} = \frac{\text{Target Dose (mg)}}{\text{Concentration Mg}} \times \text{Concentration mL}$$
3. **Daily Dose Cap Calculation:**$$\text{Daily Dose (mg)} = \text{Target Dose (mg)} \times \text{Frequency Count}$$$$\text{Final Daily Dose (mg)} = \min\left(\text{Daily Dose (mg)}, \text{Max Daily Dose (mg)}\right)$$

2.2 Safety & Red Flag Evaluation Logic Flow



```
                              v
                     +-----------------------+
                     | Calculate Dose (mg)   |---> Exceeds Max Dose Cap? ---> [ CAP DOSE
+ DISPLAY "MAX CAP" ]
                     +-----------------------+
                                 |
                                 v
                       [ Render Safe Result ]
```

3. Data Schema & Models (Dart Definition)

3.1 Patient Context Model

```
class PatientContext {
  final double weightKg;
  final int? ageMonths;
  final bool isG6PDDeficient;
  final bool hasRenalImpairment;

  PatientContext({
    required this.weightKg,
    this.ageMonths,
    this.isG6PDDeficient = false,
    this.hasRenalImpairment = false,
  });
}
```

3.2 Drug Entity Model

```
class Drug {
  final String id;
  final String genericNameEn;
  final String genericNameTh;
  final String category; // OPD, Antibiotic, ER, Asthma
  final DoseRule doseRule;
  final List<Concentration> concentrations;
  final SafetyRule safetyRule;
  final bool isPinned;

  Drug({
    required this.id,
    required this.genericNameEn,
    required this.genericNameTh,
    required this.category,
    required this.doseRule,
```

```
    required this.concentrations,
    required this.safetyRule,
    this.isPinned = false,
  });
}

class DoseRule {
  final double minMgKgDose;
  final double maxMgKgDose;
  final double stdMgKgDose;
  final String frequencyText;
  final int dosesPerDay;
  final double maxSingleDoseMg;
  final double maxDailyDoseMg;

  DoseRule({
    required this.minMgKgDose,
    required this.maxMgKgDose,
    required this.stdMgKgDose,
    required this.frequencyText,
    required this.dosesPerDay,
    required this.maxSingleDoseMg,
    required this.maxDailyDoseMg,
  });
}

class Concentration {
  final String id;
  final String label;
  final double mg;
  final double ml;
  final bool isDefault;

  Concentration({
    required this.id,
    required this.label,
    required this.mg,
    required this.ml,
    required this.isDefault,
  });
}

class SafetyRule {
  final int minAgeMonths;
  final bool g6pdSafe;
  final bool renalSafe;
  final List<String> contraindicationMessages;

  SafetyRule({
    required this.minAgeMonths,
    required this.g6pdSafe,
    required this.renalSafe,
```

```
    required this.contraindicationMessages,
  });
}
```

4. State Management Strategy (Riverpod)

To maintain reactivity without unnecessary rebuilds:

- `patientContextProvider`: Holds current weight, age, and condition toggles.
- `selectedCategoryProvider`: Holds active tab index (`OPD`, `Antibiotics`, etc.).
- `calculatedDrugListProvider`: A combined Stream/Notifier Provider that listens to `patientContextProvider` and `selectedCategoryProvider`, executing calculations on weight change in $< 10\text{ ms}$.

```
final calculatedDrugListProvider = Provider.autoDispose<List<CalculatedDrugResult>>((ref)
{
  final patient = ref.watch(patientContextProvider);
  final category = ref.watch(selectedCategoryProvider);
  final drugRepo = ref.watch(drugRepositoryProvider);

  if (patient.weightKg <= 0) return [];

  final drugs = drugRepo.getDrugsByCategory(category);
  return drugs.map((drug) => DoseCalculatorEngine.calculate(drug, patient)).toList();
});
```

5. Offline Database & Migration Strategy

1. **Local Bundled Seed Data:**
   - A predefined `drugs_v1.json` asset bundle is shipped within the app binary.
   - On first launch, the `DatabaseSeeder` populates the local NoSQL storage (Isar / Hive DB).
2. **In-Memory Caching:**
   - Active drug lists are stored in memory upon app startup to guarantee instant reactivity when typing weights.
3. **Remote Schema Updates (Optional Future Sync):**

- Check sum check against remote JSON server on internet availability to patch drug databases without requiring full app store updates.

6. Test-Driven Development (TDD) & Testing Strategy

6.1 Unit Test Coverage Target: > 95% for Domain Engine

Unit testing is vital for clinical software to avoid rounding errors or miscalculated doses.

Example Unit Test Cases (Dart / Flutter Test)

```
void main() {
  group('DoseCalculatorEngine Tests', () {
    final samplePara = Drug(
      id: 'peds_para',
      genericNameEn: 'Paracetamol',
      genericNameTh: 'พาราเซตามอล',
      category: 'OPD Common',
      doseRule: DoseRule(
        minMgKgDose: 10,
        maxMgKgDose: 15,
        stdMgKgDose: 10,
        frequencyText: 'Q 4-6 hr',
        dosesPerDay: 4,
        maxSingleDoseMg: 500,
        maxDailyDoseMg: 2000,
      ),
      concentrations: [
        Concentration(id: 'c1', label: '120mg/5mL', mg: 120, ml: 5, isDefault: true),
      ],
      safetyRule: SafetyRule(
        minAgeMonths: 1,
        g6pdSafe: true,
        renalSafe: true,
        contraindicationMessages: [],
      ),
    );

    test('Standard Weight Calculation (12kg child)', () {
      final patient = PatientContext(weightKg: 12.0);
      final result = DoseCalculatorEngine.calculate(samplePara, patient);

      // 12 kg * 10 mg/kg = 120 mg
      // 120 mg in 120mg/5mL = 5.0 mL
      expect(result.calculatedMg, equals(120.0));
```

```
      expect(result.calculatedMl, equals(5.0));
      expect(result.isMaxCapReached, isFalse);
    });

    test('Max Dose Cap Enforced (60kg patient capped at Adult Max 500mg)', () {
      final patient = PatientContext(weightKg: 60.0);
      final result = DoseCalculatorEngine.calculate(samplePara, patient);

      // 60 kg * 10 mg/kg = 600 mg -> Capped at 500 mg
      expect(result.calculatedMg, equals(500.0));
      expect(result.isMaxCapReached, isTrue);
    });

    test('Red Flag Triggered for Contraindicated Age (<2 years CPM)', () {
      final cpm = Drug(
        id: 'cpm',
        genericNameEn: 'Chlorpheniramine',
        genericNameTh: 'ซีพีเอ็ม',
        category: 'OPD Common',
        doseRule: DoseRule(minMgKgDose: 0.35, maxMgKgDose: 0.35, stdMgKgDose: 0.35, frequ
encyText: 'TID', dosesPerDay: 3, maxSingleDoseMg: 2, maxDailyDoseMg: 6),
        concentrations: [Concentration(id: 'c1', label: '2mg/5mL', mg: 2, ml: 5, isDefaul
t: true)],
        safetyRule: SafetyRule(minAgeMonths: 24, g6pdSafe: true, renalSafe: true, contrai
ndicationMessages: ['Contraindicated in children under 2 years']),
      );

      final patient = PatientContext(weightKg: 10.0, ageMonths: 14); // 1 year 2 months
      final result = DoseCalculatorEngine.calculate(cpm, patient);

      expect(result.hasRedFlag, isTrue);
      expect(result.redFlagMessage, contains('under 2 years'));
    });
  });
}
```

7. Performance Guidelines & Constraints

- **App Launch Time:** Cold start $\le 1.2\text{ s}$, Warm start $\le 0.4\text{ s}$.
- **Weight Input Throttle:** Debounce time set to `0ms` (Direct reactivity with In-memory caching).
- **Memory Footprint:** Keep under 60 MB RAM usage at all times.
- **Frame Rate:** Constant 60fps/120fps smooth scroll during continuous keyboard input.